FORM-FB-01: Fichas bibliográficas sobre Sistemas Distribuidos y Microservicios

Cesar Abrahan Macias Vasquez
Universidad Técnica Estatal de Quevedo (UTEQ)

8 de noviembre de 2026

Declaración de uso de Inteligencia Artificial

Para la realización de este trabajo se empleó un modelo de lenguaje de Inteligencia Artificial (Gemini) exclusivamente como motor de búsqueda avanzado para identificar cinco fuentes académicas reales, indexadas y recientes (2021-2026), y extraer los datos estructurados en formato BibTeX. Posterior a la búsqueda, se verificaron manualmente de forma individual todos los identificadores DOI e ISBN para garantizar que condujeran exactamente a las obras citadas en bases como IEEE Xplore, validando la autenticidad de los metadatos y evitando referencias alucinadas. Los análisis, resúmenes reflexivos y la redacción de las justificaciones son de autoría propia.

Fichas Bibliográficas

Ficha 1

Referencia: blinowski2022monolithic.
DOI verificado: `https://doi.org/10.1109/ACCESS.2022.3152803`
Resumen: El artículo aborda el problema de la elección arquitectónica al desarrollar aplicaciones modernas, contrastando las arquitecturas monolíticas con las de microservicios ante distintas cargas de tráfico. El método empleado es un estudio empírico cuantitativo, donde los autores desarrollaron una aplicación de prueba en ambos enfoques y midieron el tiempo de respuesta y la escalabilidad bajo estrés. Los resultados indican que, aunque los monolitos son más rápidos para una carga pequeña en una sola máquina debido a la ausencia de latencia de red, los microservicios presentan una escalabilidad significativamente superior en escenarios de alta concurrencia.
Dato concreto: En la evaluación de un solo nodo, la arquitectura monolítica llega a ser hasta un 40 % más rápida para cargas bajas, pero bajo alto estrés los microservicios escalan de forma lineal sin colapsar (Sección V. Evaluación de Resultados, p. 20365).
Nota personal: Esto me ayudará en el informe TA-IND-01 para fundamentar analíticamente por qué un sistema distribuido real decide asumir el costo de la latencia de red a cambio de obtener alta escalabilidad en la nube.

Ficha 2

Referencia: alelyani2024optimizing.
DOI verificado: `https://doi.org/10.1109/ACCESS.2024.3373316`
Resumen: La investigación ataca el problema del tráfico de red excesivo y la baja tolerancia a fallos en entornos de nube que orquestan microservicios bajo demandas fluctuantes. El método consiste en diseñar una novedosa estrategia algorítmica de programación, modelando los componentes mediante teoría de grafos y asignando recursos dinámicamente. Las simulaciones en infraestructuras cloud confirman que esta estrategia supera a los algoritmos tradicionales, logrando disminuir la congestión de la red inter-servicios, acortando la latencia e incrementando la tolerancia ante fallos abruptos de nodos.
Dato concreto: La estrategia propuesta logró reducir el tráfico de red interno en un 18 % y elevó la tasa de tolerancia a fallos en un 12 % al compararse con el algoritmo Round-Robin predeterminado (Sección IV. Performance Evaluation, p. 35142).
Nota personal: Su enfoque me será muy útil para la sección de tolerancia a fallos del informe TA-IND-01, demostrando cómo una red distribuida sobrevive a la caída de un servidor con una buena orquestación.

Ficha 3

Referencia: sharma2024gridappsd.
DOI verificado: `https://doi.org/10.1109/ACCESS.2024.3374331`
Resumen: El artículo presenta GridAPPS-D, una arquitectura de aplicaciones distribuidas diseñada para modernizar operaciones críticas en sistemas de distribución eléctrica. A diferencia de los sistemas monolíticos centralizados, se propone un modelo distribuido puro que permite ejecutar análisis y algoritmos de control directamente en distintos nodos físicos de la red. La metodología abarca el desarrollo de una API estandarizada. Los resultados demuestran que esta plataforma soporta eficientemente el control local, facilitando la resiliencia sistémica y evitando que el colapso de un nodo apague la red.
Dato concreto: La arquitectura estandariza el intercambio de mensajes utilizando el Çommon Information Model (CIM)"del estándar IEC 61970/61968, permitiendo que múltiples aplicaciones distribuidas operen simultáneamente (Sección III. Architecture Design, p. 39865).
Nota personal: Utilizaré este artículo en TA-IND-01 para argumentar cómo la descentralización en aplicaciones distribuidas (más allá de los microservicios web) garantiza la interoperabilidad asíncrona y previene puntos únicos de fallo.

Ficha 4

Referencia: ghosh2023distributed.
DOI/ISBN verificado: `https://doi.org/10.1002/9781119825968` (ISBN: 978-1-119-82593-7)
Resumen: Este libro detalla íntegramente los fundamentos teóricos y prácticos de los sistemas distribuidos, cerrando la brecha entre los protocolos matemáticos clásicos y el Cloud Computing. A nivel metodológico, la obra formula matemáticamente los teoremas fundamentales de la concurrencia, analizando el Teorema CAP y la tolerancia a fallas bizantinas. Constituye un texto de referencia riguroso que asienta la teoría detrás de la orquestación actual de sistemas altamente distribuidos bajo consistencia eventual.
Dato concreto: En el estudio de la sincronización temporal, detalla cómo el uso de relojes

lógicos (como los de Lamport) constituye el único método determinista para mantener el orden parcial en sistemas sin estado global (Capítulo 5, p. 91).
Nota personal: Aportará la base bibliográfica en TA-IND-01 para definir las propiedades fundacionales de los sistemas distribuidos y argumentar sobre los límites teóricos del sistema que evaluaré.

Ficha 5

Referencia: ford2021software.
ISBN verificado: 978-1-492-08689-5
Resumen: El libro (de más de 400 páginas) aborda los desafíos arquitectónicos más complejos al diseñar e implementar sistemas distribuidos modernos. Metodológicamente, los autores utilizan patrones de diseño, análisis de compensaciones (trade-offs) y casos de estudio reales para desglosar problemas como la fragmentación de datos, transacciones distribuidas y la comunicación asíncrona. Como resultado, establecen marcos de decisión formales para determinar cuándo es viable usar una arquitectura descentralizada y cómo manejar el acoplamiento entre servicios.
Dato concreto: Al analizar el manejo de datos, la obra enfatiza que .^{el} patrón Saga es el mecanismo preferido y más robusto para mantener la consistencia eventual entre múltiples microservicios sin depender de bloqueos de bases de datos centralizadas"(Capítulo 7: Managing Distributed Transactions, p. 185).
Nota personal: Usaré este libro en TA-IND-01 para evaluar críticamente los "trade-offs"(compensaciones técnicas) que tomaron los arquitectos del sistema real, especialmente en cómo resolvieron la consistencia de sus bases de datos distribuidas.